



Romper el mundo

Guía 3 · Taller 02 de caso aplicado de IA en industria

Cuatro formas de degradar la flota, cada una con una huella distinta en la telemetría. Esta guía es para seguir rompiendo cosas después del taller.

Las tres recetas

```
make seed                                # crea el mundo desde cero
make romper ESCENARIO=sesgo_silencioso # lo degrada
make reparar                             # lo devuelve a sano
```

romper y reparar son la misma receta con la primera línea cambiada: donde una aplica un escenario, la otra regenera el histórico limpio. Las otras dos líneas son idénticas.

```
romper:
    $(EN_PLATAFORMA) escenario --nombre $(ESCENARIO)
    $(MAKE) pronosticar
    $(MAKE) metricas

reparar:
    $(EN_PLATAFORMA) datos
    $(MAKE) pronosticar
    $(MAKE) metricas
```

Y lo importante es **lo que no está en ninguna de las dos: no hay entrenar**. El mundo cambia, se vuelve a correr el job con los modelos viejos, y se mide. Los 192 artefactos son los mismos todo el rato — que es literalmente lo que pasa en producción, donde reentrenar es semanal o mensual y el mundo cambia cuando le da la gana.

Los cuatro escenarios

ESCENARIO	QUÉ PASA	MAPE FLOTA	SESGO	CÓMO SE VE
(sano)	Nada	13.8%	+0.8%	La referencia
campana_promocional	Descuentos agresivos en bebidas	16.0%	-10.6%	Bebidas se despega sola, a 30.8%
sesgo_silencioso	La demanda cae despacio en toda la cadena	14.5%	+4.7%	Nada destaca. El MAPE apenas se mueve
feed_caído	Una tienda deja de reportar	13.7%	+0.8%	Anomalía en arequipa. Las métricas se ven sanas
quiebre_stock	Faltantes de stock en carnes	21.2%	+2.8%	Grita. Y no hay que reentrenar

Medidos contra el mismo mundo limpio. Los tuyos se van a parecer sin ser idénticos.

Los dos que enseñan más

sesgo_silencioso. El MAPE se mueve siete décimas —menos de lo que se mueve entre dos semanas cualesquiera— y el sesgo se multiplica por seis. Son más de 36,000 unidades de más en almacén en catorce días, y **ningún tablero con umbral sobre el MAPE suena**. Nunca.

feed_caído. Una tienda muda **no reporta ceros: no reporta nada**. Las filas desaparecen, igual que en producción.

```
# Es la diferencia entre "vendimos cero" y "no sabemos cuánto vendimos",  
# y el agente tiene que notarla.  
if nombre == "feed_caído":  
    quedan = [f for f in filas  
               if not (f["tienda"] == tienda and fecha >= muda)]
```

Esa distinción —anomalía de datos contra degradación del modelo— es la decisión más cara del taller: sobre deriva se reentrena, sobre anomalía reentrenar **rompe un modelo que estaba sano**.

Armarte escenarios propios

`escenario` acepta tres parámetros más para acotar el daño:

```
docker compose run --rm plataforma python -m plataforma escenario \
  --nombre sesgo_silencioso --categoria carnes --tienda tacna --desde 2026-07-15
```

Después, siempre: `make pronosticar && make metricas`. El comando te lo recuerda al terminar.

Cuidado con apilar escenarios. Sin un `make reparar` en medio, dos escenarios se suman y las lecturas dejan de significar nada.